

TOPIC 4: DYNAMIC PROGRAMMING

Shivani S - 192471030

#1 Dice Throw

Program

```
def dice_throw(sides, dice, target):
```

```
    dp = [[0]*(target+1)
           for _ in range(dice+1)]
```

```
    dp[0][0] = 1
```

```
    for d in range(1, dice+1):
```

```
        for t in range(1, target+1):
```

```
            for face in range(1, sides+1):
```

```
                if t-face >= 0:
```

```
                    dp[d][t] += dp[d-1][t-face]
```

```
    return dp[dice][target]
```

```
print(dice_throw(6,2,7))
```

```
print(dice_throw(4,3,10))
```

Output

6

6

#2 Assembly Line Scheduling

Program

```
def assembly(a1,a2,t1,t2,e1,e2,x1,x2):
```

```
    n=len(a1)
```

```
    f1=[0]*n
```

```
    f2=[0]*n
```

```
    f1[0]=e1+a1[0]
```

```
    f2[0]=e2+a2[0]
```

```
    for i in range(1,n):
```

```
        f1[i]=min(
            f1[i-1]+a1[i],
            f2[i-1]+t2[i-1]+a1[i]
        )
```

```
        f2[i]=min(
            f2[i-1]+a2[i],
            f1[i-1]+t1[i-1]+a2[i]
        )
```

```
    return min(
        f1[-1]+x1,
        f2[-1]+x2
    )
```

```
print(
assembly(
[4,5,3,2],
[2,10,1,4],
[7,4,5],
[9,2,8],
10,12,18,7
))
```

Output

36

#3 Minimum Production Line

Program

```
line1=[5,9,3]
```

```
line2=[6,8,4]
```

```
line3=[7,6,5]
```

```
ans=min(  
    sum(line1),  
    sum(line2),  
    sum(line3)  
)
```

```
print(ans)
```

Output

17

#4 Travelling Salesman Problem

Program

```
from itertools import permutations
```

```
mat=[  
    [0,10,15,20],  
    [10,0,35,25],  
    [15,35,0,30],  
    [20,25,30,0]  
]
```

```
n=4
```

```
ans=9999

for p in permutations(range(1,n)):

    cost=0

    k=0

    for j in p:

        cost+=mat[k][j]

        k=j

    cost+=mat[k][0]

    ans=min(ans,cost)

print(ans)
```

Output

80

#5 Cities Permutation

Program

```
from itertools import permutations

cities=['A','B','C','D','E']

for path in permutations(cities):

    print(path)

    break
```

Output

('A', 'B', 'C', 'D', 'E')

#6 Longest Palindromic Substring

Program

```
def longest_palindrome(s):  
  
    result = ""  
  
    for i in range(len(s)):  
  
        for j in range(i+1, len(s)+1):  
  
            temp = s[i:j]  
  
            if temp == temp[::-1]:  
  
                if len(temp) > len(result):  
  
                    result = temp  
  
    return result  
  
print(longest_palindrome("babad"))  
  
print(longest_palindrome("cbbd"))
```

Output

bab
bb

#7 Longest Substring Without Repeating Characters

Program

```
def longest_substring(s):  
  
    seen = ""
```

```
ans = 0
```

```
for ch in s:
```

```
    while ch in seen:
```

```
        seen = seen[1:]
```

```
    seen += ch
```

```
    ans = max(
```

```
        ans,
```

```
        len(seen)
```

```
    )
```

```
return ans
```

```
print(
```

```
    longest_substring(
```

```
        "abcabcbb"
```

```
    ))
```

```
print(
```

```
    longest_substring(
```

```
        "bbbbbb"
```

```
    ))
```

```
print(
```

```
    longest_substring(
```

```
        "pwwkew"
```

```
    ))
```

Output

```
3
```

```
1
```

```
3
```

#8 Word Break

Program

```
def word_break(s, words):
```

```
    dp = [False]*(len(s)+1)
```

```
    dp[0] = True
```

```
    for i in range(
```

```
        1,
```

```
        len(s)+1
```

```
    ):
```

```
        for j in range(i):
```

```
            if (
```

```
                dp[j]
```

```
                and
```

```
                s[j:i] in words
```

```
            ):
```

```
                dp[i] = True
```

```
    return dp[-1]
```

```
print(
```

```
word_break(
```

```
"leetcode",
```

```
["leet","code"]
```

```
)
```

Output

True

#9 Dictionary Word Break

Program

```
dictionary = [  
    "i",  
    "like",  
    "sam",  
    "sung",  
    "samsung"  
]
```

```
def check(s):
```

```
    for word in dictionary:
```

```
        s = s.replace(  
            word,  
            ""  
        )
```

```
    return s == ""
```

```
print(  
    check(  
        "ilike"  
    ))
```

```
print(  
    check(  
        "ilikesamsung"  
    ))
```

Output

True

True

#10 Text Justification

Program

```
def justify(words, width):
```

```
    line = ""
```

```
    for w in words:
```

```
        if (
            len(line)
            +
            len(w)
            +
            1
            <=
            width
        ):
```

```
            line += w + " "
```

```
    else:
```

```
        print(
            line.ljust(
                width
            )
        )
```

```
        line = w + " "
```

```
    print(
        line.ljust(
            width
        )
    )
```

```
justify(  
[  
"This",  
"is",  
"an",  
"example",  
"of",  
"text",  
"justification"  
],  
16  
)
```

Output

This is an
example of text
justification

#11 Word Filter

Program

```
class WordFilter:
```

```
    def __init__(self, words):
```

```
        self.words = words
```

```
    def f(self, pref, suff):
```

```
        for i in range(  
            len(self.words)-1,  
            -1,  
            -1  
        ):
```

```
            if (  
                self.words[i]
```

```
        .startswith(pref)

    and

    self.words[i]
    .endswith(suff)
):

    return i

return -1
```

```
obj = WordFilter(
["apple"]
)
```

```
print(
obj.f(
"a",
"e"
))
```

Output

0

#12 Floyd Warshall Algorithm

Program

```
INF = 999
```

```
graph = [
[0,3,INF,-4],
[INF,0,4,1],
[2,INF,0,INF],
[INF,6,-5,0]
]
```

```

n = len(graph)

for k in range(n):

    for i in range(n):

        for j in range(n):

            graph[i][j] = min(
                graph[i][j],
                graph[i][k]
                +
                graph[k][j]
            )

for row in graph:

```

```

    print(row)

```

Output

```

[-2, 2, -9, -6]
[5, 0, -4, 1]
[2, 5, 0, -2]
[-3, 2, -5, -8]

```

#13 Floyd Algorithm – Network Failure

Program

```

INF = 99999

```

```

graph = [
    [0,1,5,INF,INF,INF],
    [1,0,2,1,INF,INF],
    [5,2,0,INF,3,INF],
    [INF,1,INF,0,1,6],
    [INF,INF,3,1,0,2],

```

```
[INF,INF,INF,6,2,0]  
]
```

```
def floyd(graph):
```

```
    n=len(graph)
```

```
    dist=[row[:] for row in graph]
```

```
    for k in range(n):
```

```
        for i in range(n):
```

```
            for j in range(n):
```

```
                dist[i][j]=min(  
                    dist[i][j],  
                    dist[i][k]  
                    +  
                    dist[k][j]  
                )
```

```
    return dist
```

```
result=floyd(graph)
```

```
print("Before Failure:")
```

```
print(  
    "Router A to F =",  
    result[0][5]  
)
```

```
graph[1][3]=INF
```

```
graph[3][1]=INF
```

```
result=floyd(graph)
```

```
print("After Failure:")
print(
"Router A to F =",
result[0][5]
)
```

Output

Before Failure:
Router A to F = 6

After Failure:
Router A to F = 10

#14 Distance Matrix

Program

```
INF = 99999
```

```
graph = [
[0,2,INF,INF,8],
[2,0,3,INF,2],
[INF,3,0,1,INF],
[INF,INF,1,0,1],
[8,2,INF,1,0]
]
```

```
n=len(graph)
```

```
for k in range(n):
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            graph[i][j]=min(
                graph[i][j],
                graph[i][k]
```

```

        +
        graph[k][j]
    )

print(
    "Distance Matrix"
)

```

for row in graph:

```
    print(row)
```

Output

Distance Matrix

[0, 2, 5, 5, 4]

[2, 0, 3, 3, 2]

[5, 3, 0, 1, 2]

[5, 3, 1, 0, 1]

[4, 2, 2, 1, 0]

#15 Optimal Binary Search Tree

Program

```
def obst(freq):
```

```
    n=len(freq)
```

```
    cost=[
```

```
        [0]*n
```

```
        for _ in range(n)
```

```
    ]
```

```
    for i in range(n):
```

```
        cost[i][i]=freq[i]
```

```
    for length in range(2,n+1):
```

```

for i in range(n-length+1):

    j=i+length-1

    cost[i][j]=float(
        "inf"
    )

    total=sum(
        freq[i:j+1]
    )

    for r in range(i,j+1):

        c=total

        if r>i:

            c+=cost[i][r-1]

        if r<j:

            c+=cost[r+1][j]

        cost[i][j]=min(
            cost[i][j],
            c
        )

return cost[0][n-1]

```

```

freq=[
0.1,
0.2,
0.4,
0.3

```



```
]
```

```
print(  
"Cost =",  
obst(freq)  
)
```

Output

Cost = 1.7

#16 Optimal Binary Search Tree

Program

```
def obst(freq):  
  
    n = len(freq)  
  
    cost = [[0]*n for _ in range(n)]  
  
    for i in range(n):  
  
        cost[i][i] = freq[i]  
  
    for length in range(2, n+1):  
  
        for i in range(n-length+1):  
  
            j = i + length - 1  
  
            cost[i][j] = float("inf")  
  
            total = sum(freq[i:j+1])  
  
            for r in range(i, j+1):  
  
                c = total  
  
                if r > i:
```

```
    c += cost[i][r-1]
```

```
    if r < j:
```

```
        c += cost[r+1][j]
```

```
    cost[i][j] = min(  
        cost[i][j],  
        c  
    )
```

```
    return cost[0][n-1]
```

```
freq = [0.1,0.2,0.4,0.3]
```

```
print("Cost =", obst(freq))
```

Output

Cost = 1.7

#17 Longest Common Subsequence (LCS)

Program

```
def lcs(a, b):
```

```
    m = len(a)
```

```
    n = len(b)
```

```
    dp = [  
        [0]*(n+1)  
        for _ in range(m+1)  
    ]
```

```
    for i in range(1, m+1):
```

```
        for j in range(1, n+1):
```

```

        if a[i-1] == b[j-1]:

            dp[i][j] = (
                dp[i-1][j-1]
                + 1
            )

        else:

            dp[i][j] = max(
                dp[i-1][j],
                dp[i][j-1]
            )

    return dp[m][n]

print(
lcs(
"ABCB DAB",
"BDCAB"
))

```

Output

4

#18 Knapsack Problem

Program

```

def knapsack(w,v,c):

    n = len(w)

    dp = [
        [0]*(c+1)
        for _ in range(n+1)
    ]

```

```

for i in range(1,n+1):

    for j in range(c+1):

        if w[i-1] <= j:

            dp[i][j] = max(
                v[i-1]
                +
                dp[i-1][j-w[i-1]],

                dp[i-1][j]
            )

        else:

            dp[i][j] = dp[i-1][j]

    return dp[n][c]

```

```

print(
    knapsack(
        [2,3,4],
        [4,5,6],
        5
    ))

```

Output

9

#19 Coin Change

Program

```

def coin_change(coins, amount):

```

```
dp = [999]*(amount+1)
```

```
dp[0] = 0
```

```
for i in range(
    1,
    amount+1
):
```

```
    for c in coins:
```

```
        if i-c >= 0:
```

```
            dp[i] = min(
                dp[i],
                dp[i-c]+1
            )
```

```
return dp[amount]
```

```
print(
    coin_change(
        [1,2,5],
        11
    ))
```

Output

3

#20 Fibonacci Series

Program

```
def fib(n):
```

```
    a = 0
```

```
    b = 1
```

```
for i in range(n):
```

```
    print(  
        a,  
        end=" "  
    )
```

```
    a, b = b, a+b
```

```
fib(10)
```

Output

```
0 1 1 2 3 5 8 13 21 34
```

#21 Rod Cutting

Program

```
def rod(price, n):
```

```
    dp = [0]*(n+1)
```

```
    for i in range(1,n+1):
```

```
        for j in range(i):
```

```
            dp[i] = max(  
                dp[i],  
                price[j]  
                +  
                dp[i-j-1]  
            )
```

```
    return dp[n]
```

```
print(  
rod(  
[1,5,8,9],  
4  
)
```

Output

10

#22 Edit Distance

Program

```
def edit(a,b):  
  
    m = len(a)  
    n = len(b)  
  
    dp = [  
        [0]*(n+1)  
        for _ in range(m+1)  
    ]  
  
    for i in range(m+1):  
  
        for j in range(n+1):  
  
            if i == 0:  
  
                dp[i][j] = j  
  
            elif j == 0:  
  
                dp[i][j] = i  
  
            elif a[i-1] == b[j-1]:  
  
                dp[i][j] = dp[i-1][j-1]
```

else:

```
    dp[i][j] = 1 + min(  
        dp[i-1][j],  
        dp[i][j-1],  
        dp[i-1][j-1]  
    )
```

```
return dp[m][n]
```

```
print(  
    edit(  
        "cat",  
        "cut"  
    ))
```

Output

1